PIRATE HUNTER

Evidence before every label.

LabelGuard
Packaged Food Label Compliance Check Service

FINAL PROJECT REPORT / MINI PROJECT 10

ML System Design & AgentOps (UE24AM342AA5)
Department of CSE (AI & ML), PES University
27 October 2026

Team member	SRN
Naveen Prasad M	PES1UG24AM379
Nishanth Nag	PES1UG24AM381
Sateesh	PES1UG24AM397

Advisory findings with exact regulatory evidence.
A regulatory affairs professional decides whether to print.
Measured profile: local deterministic planner stub.

1. Problem, scope and objective

A food manufacturer needs evidence about proposed label declarations before printing. The difficult dependency is that nutrition declarations are governed by Labelling and Display, while conditions attached to claims such as source of protein are in the separate Advertising and Claims regulation. Similarity to a nutrition-panel question alone is not enough to retrieve that second provision.

We implement a production-style asynchronous service, scoped to domestic retail processed breakfast cereals. It validates structured label/package data, chooses retrieval tools, produces requirement findings, independently verifies them and routes the run. It does not read artwork or authorise printing. The required clear-for-print route is an advisory code for supported checks; print_authorized is always false and the UI states that a professional must decide.

Imports, non-retail food, special populations, supplements, small-pack or product exemptions, unknown claims and incomplete benchmarks/evidence require human review. Sugar and fibre content claims are implemented as additional narrow rules; this is not an exhaustive compliance engine.

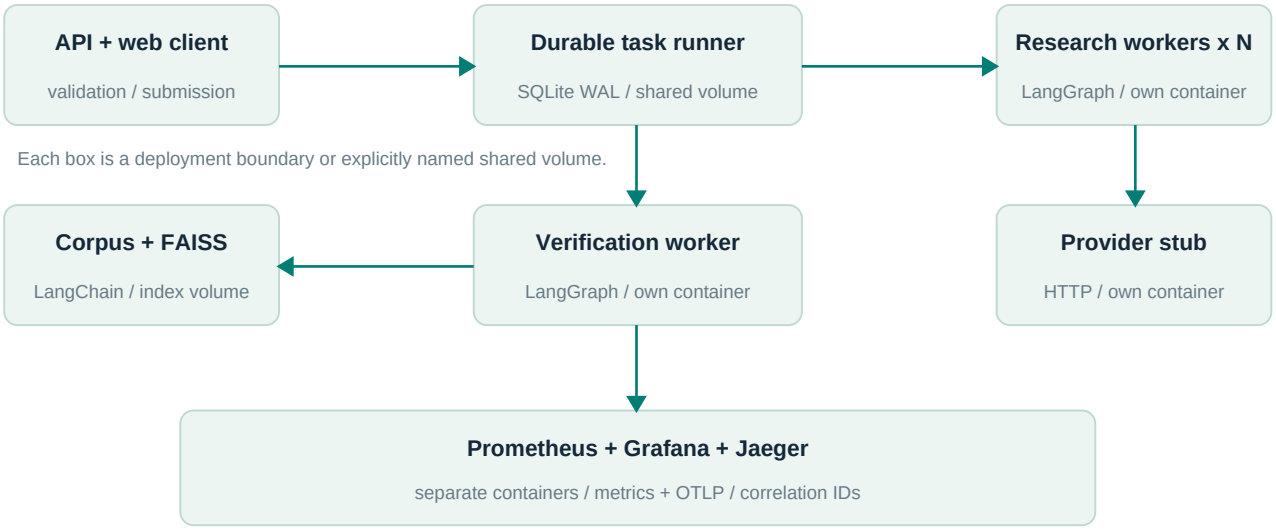
2. Corpus and reference answer

Official PDFs are bundled, hash-pinned and parsed locally: Labelling and Display 2020 version VIII (9 September 2025), Advertising and Claims 2018 version IV (14 December 2022), and the notified Labelling and Display First Amendment 2026. The latter's regulation 1(2), page 5, states commencement on 1 July 2027. This implementation escalates assessment dates at or beyond that date. Compendia are reference compilations; Gazette notifications prevail in a conflict. A full expert amendment-completeness review is not claimed.

The indexed corpus contains 146 chunks and 19 curated assessed clauses. LD-5.3 resolves to 5(3)(b), pages 10-11; AC-5.3 resolves to 5(3), page 4; AC-PROTEIN resolves to Schedule I item 11, page 9; AC-RDA resolves to 2(1)(n), page 2. A source version, source SHA256, clause, page and exact text accompany every citation. Provision anchors must resolve or the ingestion pipeline fails.

For the illustrative supplied 54 g/day benchmark, the solid-food source-of-protein alternatives are 5.4 g/100g OR 2.7 g/100kcal OR 5.4 g/serving. At 6 g/100g, 376 kcal/100g and 40 g/serve the mass branch passes; at 3 g/100g all branches fail. A 4 g/100g product at 120 kcal/100g passes the energy alternative; the same protein at a 150 g serving passes the serving alternative. The newer version-IV claims schedule includes the serving branch, which the initially located older compendium did not contain. The RDA is supplied and explicitly conditional; without reviewer confirmation it does not become an assumed universal value. Laboratory substantiation and target-population suitability remain professional responsibilities. Faculty confirmation of the reference answer is still a human Evaluation 1 activity, not something this report fabricates.

3. Agent design and deployment architecture



The client submits to a FastAPI container. Boundary validation, injection/toxicity screening and per-peer rate limiting occur before a run is inserted. Accepted requests immediately receive an ID. A persistent SQLite WAL task runner sits between submission and execution; it is not an in-process background task. Polling is chosen because the caller needs no public callback endpoint or secret.

The research agent runs in agent-worker containers. A LangGraph plan/tool loop allows an HTTP-configured planner to choose corpus_search, clause_lookup or request_fields. It reads prior observations and retrieved provision links to decide whether another regulation modifies the initial evidence. The final candidate and its passages move through an atomic durable hand-off transaction to a distinct verifier-worker process/container with a second LangGraph graph.

The verification agent re-resolves every passage against the pinned corpus and independently recomputes rule findings, nutrient arithmetic and citation metadata. It rejects unsupported candidates or source tampering, performs one repair pass with a different retrieval strategy, and otherwise escalates. It does not ask the same model to judge itself. If either process dies, leases allow bounded reclaim. Owner/lease checks block stale publication. Exhausted jobs fail with no confident result. A normal error produces a service-reliability review finding.

Deployment boundaries: API/client; scalable research workers; independent verification worker; provider stub; Prometheus; Grafana; Jaeger; plus a one-shot initialiser. A shared state volume holds queues/runs/events and an index volume holds immutable read data. The Mermaid deployment diagram and full decisions are in ARCHITECTURE.md. There are seven long-running services; initial startup used new named volumes and all became healthy.

Bounds are 24 research steps, 12 research tool calls, one verifier repair pass, 5 s per tool, 8 s per provider attempt, two retries and a 45 s submission-to-result budget. Retries back off 200/400 ms. Three failures open a per-worker circuit for 20 s. Expired/incomplete work cannot return a confident determination.

4. Retrieval, prompts, guardrails and configuration

LangChain invokes FAISS over normalised local word unigram/bigram hashing embeddings (2,048 dimensions). This is lexical, not learned semantic retrieval. The small corpus and offline reproducibility motivate the baseline. Exact clause dependencies, not unsupported semantic-retrieval claims, resolve the cross-regulation case. Clauses are individual chunks; full page context uses 1,500-character windows with 180-character overlap. Chunk/embedding hashes let changed records alone be re-embedded. A matching manifest reuses the index; FAISS binary plus JSON avoid unsafe pickle loading. Freshness and throughput are published in the corpus API and Prometheus metrics.

Prompts v1/v2 and verifier-v1 are versioned files. Each planner response records version, contents digest, context, chosen plan, usage, configured price and cost. The deliberate change is v2 dependency-directed retrieval into claims provisions; v1 prefers a labelling-only search and stops with incomplete claim evidence. Both use the same independent verifier. Cache keys include request, source/index version, prompt content/version, verifier policy and provider configuration.

Every free-text input is screened after Unicode normalisation. Tool names, argument types and read-only field paths are allowlisted. Output cannot set print authority true or emit approval language. Local pattern-based injection/toxicity detectors are limited and can be bypassed; they are not portrayed as a complete safety classifier. Schema/range/category/cross-field errors are quarantined and do not start an agent. Missing business declarations become unmet findings.

External provider URL, model identifier, key and prices are runtime settings; no external account/model is embedded. The checked-in profile is a clearly labelled deterministic HTTP planner stub with fixed rules and a 350 ms delay. It permits a repeatable complete service and scaling demo without a key. External-provider JSON behavior, model quality and paid costs were not measured.

5. Fixed-set evaluation: before and after

There are 32 fixed cases: 27 answer scenarios plus 5 boundary/guardrail rejects. Gold outcomes and supporting passage IDs are hand-authored in eval/scenarios.json. The CLI runs both actual LangGraph graphs with an in-process deterministic planner fixture, no provider delay and an isolated temporary state store. It is a reproducible equivalent evaluator, not a TruLens installation or an independent expert/model judgement.

Metric	v1 before	v2 after
groundedness	1.0000	1.0000
context relevance	0.8522	1.0000
faithfulness	1.0000	1.0000
hallucination rate	0.0000	0.0000
context recall	0.7541	1.0000
Outcome accuracy (all cases)	0.4375	1.0000

v1 matched 14/32; v2 matched 32/32, an accuracy increase of 56.25 percentage points. The baseline's incomplete evidence safely escalates rather than fabricating a conclusion. The provenance and policy-consistency metrics therefore remain at their safe ceiling while evidence relevance/coverage and useful outcome accuracy improve. No improvement is invented for a metric that did not change.

Definitions: groundedness is the fraction of met/unmet assertions with nonempty, exactly resolved citations. Context relevance is retrieved applicable curated provisions divided by all retrieved passages; relevant IDs are the union of the scenario's gold and the implemented applicable checklist. Faithfulness is the fraction of findings identical to deterministic independent policy recomputation. Hallucination rate is unsupported-citation met/unmet assertions divided by all met/unmet assertions. Context recall is retrieved applicable IDs divided by required/gold IDs. Scenario-level metric means exclude rejected inputs; outcome accuracy includes them. An all-escalation answer has no positive assertions and can score 1 groundedness/0 hallucinations, which is why accuracy and context recall are reported too. These metrics share the implemented policy and do not establish independent legal truth or unseen-case/model generalisation.

6. Complete-run SLO, load and scaling measurements

Before measuring, docs/SLO.md declared p95 <= 15 s for submission -> final verified result in the stub profile. An analyst's interactive pre-print workflow can wait seconds for evidence, but a growing queue must be visible. The hard run budget is independently 45 s. Changing the target after observing results is rejected.

The actual containerised Locust sweep uses cache-disabled qualifying requests, 350 ms planner responses, 30 s arrival periods and up to 50 s drain time. Batch identifiers prevent cache collisions. The RUN/submission_to_result transaction, not POST timing, supplies these metrics. Counts/quantiles are from saved Locust CSV snapshots; Locust rounds histogram buckets and may emit its last in-flight completion after the last periodic CSV write. Samples are small, and the closed workload plus drain period make RPS a local observation rather than a universal maximum capacity claim.

Workers	Users	Runs	RPS	p50 s	p95 s	Failed/review	15 s SLO
1	2	11	0.332	5.60	6.20	0	met
1	4	13	0.333	11.00	12.00	0	met
1	8	17	0.321	22.00	24.00	0	missed
3	4	31	0.963	3.40	5.50	0	met
3	8	36	0.966	8.30	8.90	0	met
3	20	46	0.993	18.00	20.00	0	missed

1 research worker(s): first tested miss at 8 concurrent users; 3 research worker(s): first tested miss at 20 concurrent users. These are the first failing points in the tested grid, not precise saturation thresholds. At eight concurrent users, three research workers deliver 3.01 times the observed throughput of one worker. The dominant bottleneck in this range is waiting/queueing at the research stage, which makes multiple serial provider calls; verification is short and remains a separate consumer. SQLite's single writer can become a later bottleneck, but that is not presented as a measured saturation result. Longer tests, finer concurrency sweeps and resource isolation are needed for capacity planning.

7. Caching, rate limits, usage and cost

Identical verified whole-run results have a one-hour TTL cache. Entries are invalidated by request/corpus/prompt/policy/provider changes. Incomplete service or evidence results are not cached. Every hit gets a new run record and an explicit hit flag. Per-peer rate limiting defaults to 30 write requests/minute; the load-only override raises it so system capacity can be tested independently.

Ten sequential complete HTTP runs per condition compare disabled caching with an already-primed cache. The priming request is excluded. Client-observed p95 is the nearest-rank maximum for n=10, so this is a small demonstration, not a stable tail estimate.

Condition	p50 s	p95 s	Model calls/run	Actual cost/run	Cost/100 runs
cache disabled	3.1560	4.4210	7.0	0.000000	0.000000
warm cache	0.0390	0.0470	0.0	0.000000	0.000000

Observed p95 falls by 98.94%. Actual provider cost is zero in both conditions, so no dollar saving can be claimed. Calls fall to zero on cache hits. Stub token counts are explicitly character/4 estimates, not a model tokenizer. External successful calls record provider-reported prompt/completion tokens and the configured prices at call time. If a failed external call returns no usage, its billed cost is unknown; the system does not fabricate that information. The dominant potential cost driver is repeated research-planner context after retrieval: full evidence is sent on successive planning calls. Pricing a real provider requires its actual configured rates and reported usage, which were not available in this development run.

8. Verification, observability and operational evidence

Automated tests passed for invalid-input quarantine/no job creation, numeric and boolean types, cross-fields, nested injection, maximum body size, safe output, tampered candidate/passages, expired runs, lease recovery, stale owner protection, provider retries and circuit behavior. Actual HTTP verification produced the expected protein pass/fail and human-review routes; invalid/injected requests left run counts unchanged and feedback persisted. Browser acceptance checked render, submit, poll, result, citations, activity, corpus, team and mobile overflow.

A deliberate unsupported candidate produced a failed verification event, then repair and a passing event. A real stub outage exercise opened the breaker and the next run escalated through the open-circuit path. These are labelled injected tests. The genuine registry bootstrap and missing browser-runtime failures are recorded separately in INCIDENTS.md, with observed evidence and recovery.

W3C trace context connects API, research, provider stub and verification; an actual saved Jaeger trace spans all four services. Structured logs and durable events include correlation/run IDs. Prometheus scrapes each research replica by DNS discovery, the application and the monitoring services. Grafana has 16 provisioned panels. Default application CPU/RAM are process-level proxies within their containers; optional host-specific cAdvisor covers broader container resources. Per-agent outcomes/steps/tools/latency/retries/usage, worker uptime, feedback aggregates and sampled quality history are visible in the web client. Jaeger all-in-one traces are volatile across its restarts; SQLite audit events remain durable. The runbook covers provider failure and stalled/dead workers.

9. Engineering judgement, limitations and submission

We reject a single fixed vector search, unchecked model verdicts, two prompts in one process, paid embeddings and label-image OCR. We choose separate agents with typed read-only tools, exact evidence verification, local indexing and a single-host transactional task runner. SQLite makes a clean-clone demo simple; it is not a multi-host queue. The stable lock file, Compose health dependencies, CI recipe, exported contracts, examples and incident/runbook material make the system reviewable and reproducible.

What did not work or remains unconfirmed: the initial registry pull failed transiently; the bundled browser package lacked its expected Chromium binary; v1 could not retrieve enough claim evidence; initial source discovery located an older claims compendium whose serving alternative was incomplete. Recovery and the verified newer source are documented. No real external LLM calibration, expert legal validation, exhaustive claim/ingredient catalogue, authentication, public deployment, long-duration soak test or multi-host throughput is claimed. Labelling artwork, layout dimensions, laboratory evidence, licence authenticity and detailed Legal Metrology rules still need professional review.

The repository is prepared locally for submission; a GitHub destination and faculty sign-off were not supplied and are not invented. Team identities follow the user's correction and attached roster. Individual historical contributions are not fabricated. README, REQUIREMENTS, DEMO, RUNBOOK, INCIDENTS and the saved raw measurements are companion deliverables for the live demonstration/viva.

References

- FSSAI, Labelling and Display Regulations 2020, version VIII, 9 September 2025. Official PDF and source hash: data/sources.json (labelling). - FSSAI, Advertising and Claims Regulations 2018, version IV, 14 December 2022. Official PDF and source hash: data/sources.json (claims). - FSSAI, Labelling and Display First Amendment Regulations 2026, 24 March 2026. Official Gazette PDF and source hash: data/sources.json (amendment_2026). - Course assignment: MiniProject_10_Food_Label_Compliance.pdf, supplied locally. - Reproducible evidence: output/evaluation/results.json, output/load/results.json, output/load/cache/results.json and output/verification/.